- **Domain Naming Master:** Controls the process of adding or removing entire domains. This role also applies across the whole forest.
- **RID Master:** Assigns unique ID number pools to Domain Controllers, which they then use to create new objects. This role applies at the domain level.
- **PDC Emulator:** Handles time synchronization across the network, password changes, and Group Policy processing. This role applies at the domain level.
- **Infrastructure Master:** Synchronizes object information correctly across different domains. This role also applies at the domain level.

DNS Concept & Architecture

For a helpful video explanation of how a DNS server actually works, see this reference: How DNS Server Works (YouTube)

1. The Main DNS Server Inside a Company (Local DNS)

Most companies do not rely on external or public DNS servers as their main, primary choice. Instead, they choose to run their own DNS service directly inside their Active Directory Domain Controller (DC).

Whenever any computer on the company network automatically receives its IP configuration through DHCP, the IP address of this same Active Directory server is also assigned to it, to be used as that computer's primary DNS server.

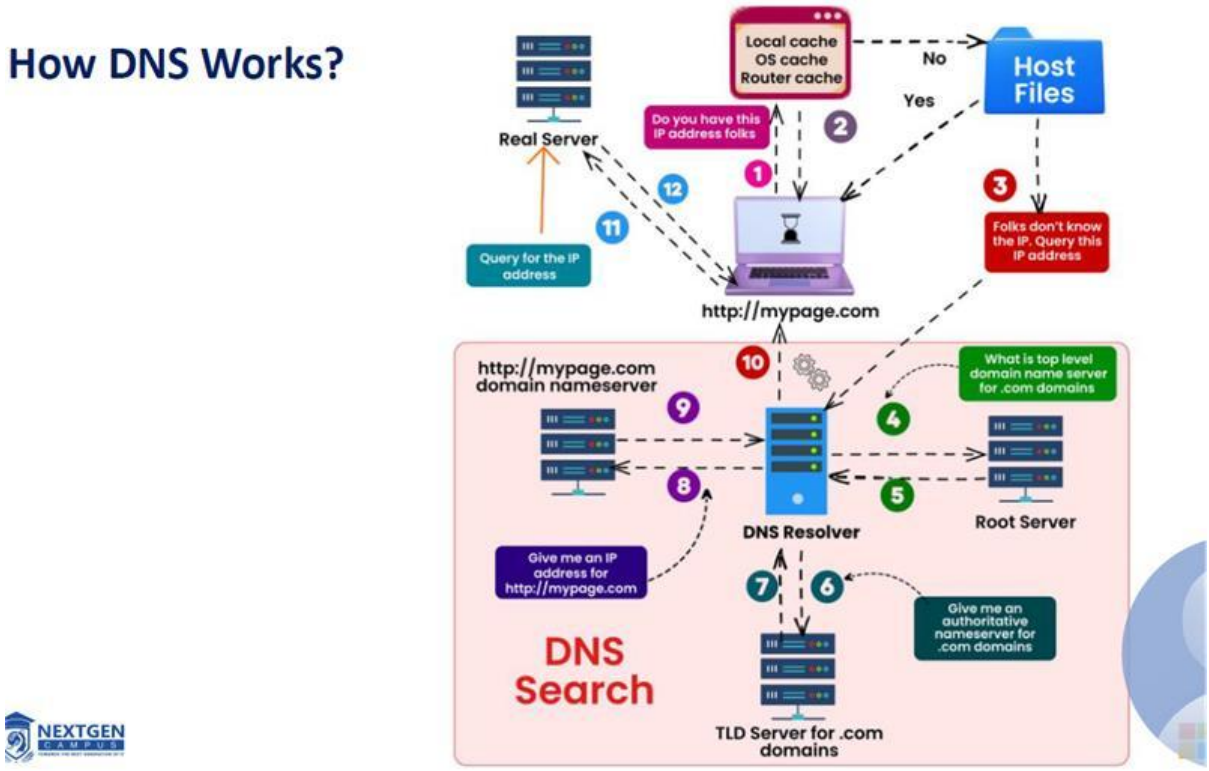
2. Searching for an Internal Website or Server (Internal Query)

Let's imagine we are working in the IT department, and we want to access the company's internal HR portal, at an address like hr.company.local.

- Our computer asks the internal DNS server directly: "Where is hr.company.local located?"
- This internal DNS server keeps a complete list of IP addresses, for every single PC and server in the company, stored inside its own database. This database is known as the Forward Lookup Zone.
- The server then instantly returns the exact matching IP address (for example: 192.168.20.5) back to our computer, and the internal website opens up successfully for us.

How DNS Works (Full Process)

When we type a website address into our browser, a longer background process actually takes place, involving several different servers working together, before the page finally loads.



As shown in the diagram above, here is a simplified version of what happens, step by step:

- First, our computer checks its own local cache, OS cache, and router cache, to see if it already knows the IP address for the website we typed in. If it already knows the answer, it uses that immediately, without going any further.
- If our computer does not already know the answer, it checks a local list called the Host Files. If that also does not contain the answer, our computer sends the request onward to a DNS Resolver.
- The DNS Resolver then works through a series of servers to find the answer: it may ask a Root Server, which points it toward the correct Top Level Domain (TLD) Server (for example, the server responsible for all ".com" domains).
- The TLD Server then points the resolver toward the correct authoritative domain nameserver, which is the specific server that actually knows the exact IP address for the website being requested.
- Once the DNS Resolver receives this final IP address, it passes that information back to our computer.
- Finally, our computer uses this IP address to contact the real web server directly, and the website is loaded in our browser.

Even though this entire process sounds long, it actually all happens within a very small fraction of a second, in the background, every single time we visit a new website.

DNS Zones

- **Primary Zone:** This zone holds the master, original copy of all DNS records, and it supports both reading and writing (making changes) to those records.
- **Secondary Zone:** This is a read-only copy of the DNS records, which is regularly updated from the Primary Zone, through a process called zone transfer.

- **Stub Zone:** This zone contains only NS (Name Server) records, and its main purpose is to help resolve names correctly across different domains.
- **Forward Lookup Zone:** This zone is used to map a website or hostname to its correct IP address (Name to IP).
- **Reverse Lookup Zone:** This zone works in the opposite direction, mapping an IP address back to its correct hostname (IP to Name).

DNS Record Types

- **A Record:** Maps a hostname directly to an IPv4 address. For example: app.bank.lk → 192.168.1.10
- **AAAA Record:** Maps a hostname directly to an IPv6 address. For example: server.bank.lk → 2001:db8::5
- **CNAME Record:** Acts as an alias, pointing to another existing record. For example, mail.bank.lk might actually just be a link that points to the main name, exchange.bank.lk.
- **PTR Record:** Used for reverse lookups. For example: 192.168.1.10 → app.bank.lk
- **MX Record:** Identifies the Mail Exchange server responsible for handling email. For example, mail.bank.lk is the server that is told to handle incoming mail whenever someone sends an email to info@bank.lk.
- **SRV Record:** Helps Active Directory locate specific Domain Controllers and services, such as LDAP. This record type is extremely important for Active Directory, because whenever a computer logs into the network, it reads this SRV record specifically, in order to find out exactly where the Domain Controller (DC) is located.

Implementing AD Server and Client in Hyper-V: Creating OUs, Users, and Connecting a Client to AD, Then Assigning a Wallpaper Policy

Active Directory: First Important Step

When we are setting up Active Directory for the very first time, there is an important security step we must do right away: we should either rename the built-in "Administrator" account, or completely disable it, and always use a strong, complex password. This is important because the Active Directory administrator account has the highest level of privilege in the entire network, meaning it has full control over everything.

Why rename or disable the Administrator account?

Every single Windows Server comes with a built-in account, which is automatically named "Administrator." Since this name is always the same by default, and well known to attackers, hackers can easily guess that this account exists, and then try to attack it directly. By either renaming this account to something less obvious, or disabling it completely and using a different account instead, we make it much harder for hackers to guess our login details.

Why an Internal Switch Was Chosen for the Active Directory Lab

When we set up Active Directory inside a virtual environment using Hyper-V, choosing the correct type of virtual switch is extremely important, for both network isolation and overall stability. For this particular project, an Internal Switch was chosen, instead of an External Switch, a Private Switch, or the Default Switch. Here is the reasoning behind that choice, explained in detail:

1. Isolation from the Production Network (compared to an External Switch)

An Active Directory environment needs its own dedicated infrastructure services, such as DHCP and DNS, running specifically for the lab. If we were to use an External Switch instead, our virtual Domain Controller would actually be connected to the same physical network as our real router and other real devices. This could cause serious conflicts, such as IP address duplication, and it could even disrupt the internet connection for other real devices on the same physical network.

By using an Internal Switch instead, our entire lab environment is completely isolated from the physical, real-world network, which means we can safely experiment and test things, without any risk of interfering with other devices.

2. Host-to-VM Communication (compared to a Private Switch)

A Private Switch does provide 100% isolation, meaning it is extremely secure. However, this also means that it completely blocks the physical host computer (our actual laptop) from being able to communicate with the virtual machines at all.

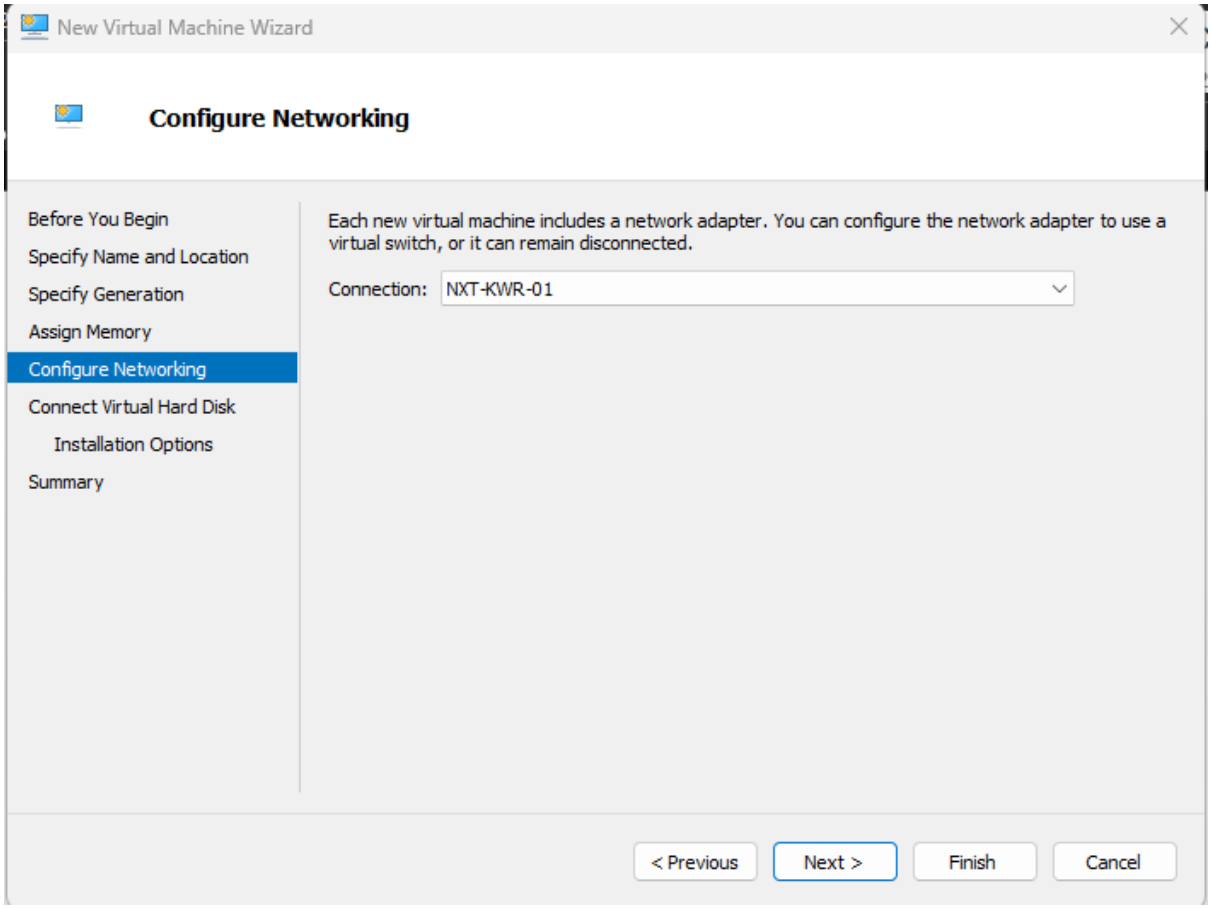
An Internal Switch, on the other hand, still allows the host computer to communicate with the virtual machines, even though the lab remains isolated from the outside network. This makes it much easier for us to transfer installation files, ISO images, and other useful tools directly from our laptop into the virtual servers, which would not be possible with a fully isolated Private Switch.

3. IP Address Stability (compared to the Default Switch)

The Hyper-V Default Switch automatically assigns dynamic IP addresses to virtual machines, and these IP addresses can actually change every single time the host computer restarts.

However, Active Directory and Domain Controllers strictly require a fixed, permanent Static IP address, in order to function correctly and reliably. If the IP address were to change unexpectedly after a restart, it could break the entire Active Directory setup.

By using an Internal Switch instead, we are able to manually configure a permanent, stable Static IP address for our lab environment, one that will not change or break, even after the host computer is restarted.

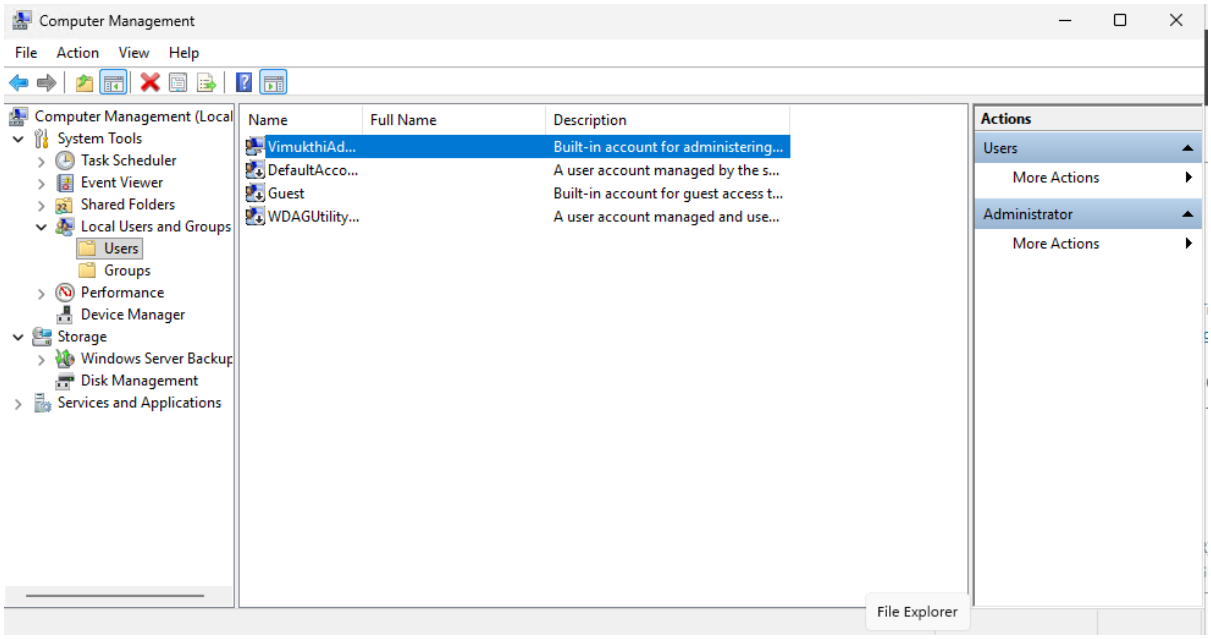


How to Rename the Administrator Username

To rename the Administrator account on our server, we can follow this simple path:

1. Open **Tools**.
2. Go to **Computer Management**.
3. Select **Local Users and Groups**.
4. Click on **Users**.

Once we are inside the Users folder, we can find the built-in "Administrator" account in the list, right-click on it, and rename it to something less obvious, so it becomes harder for anyone to guess.

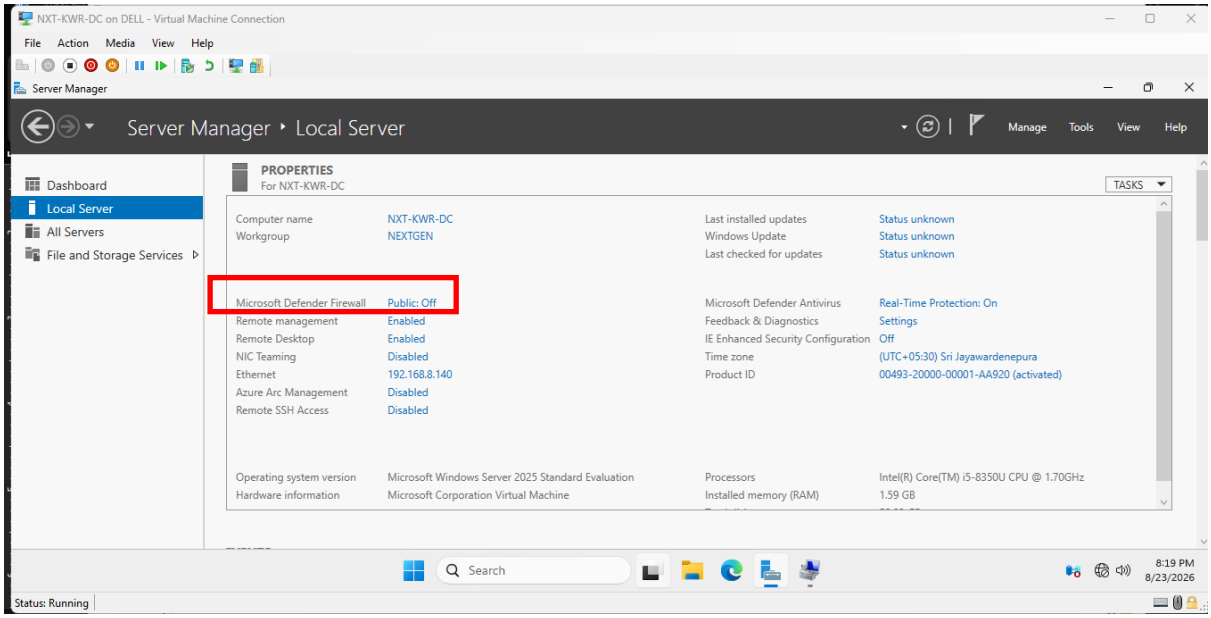


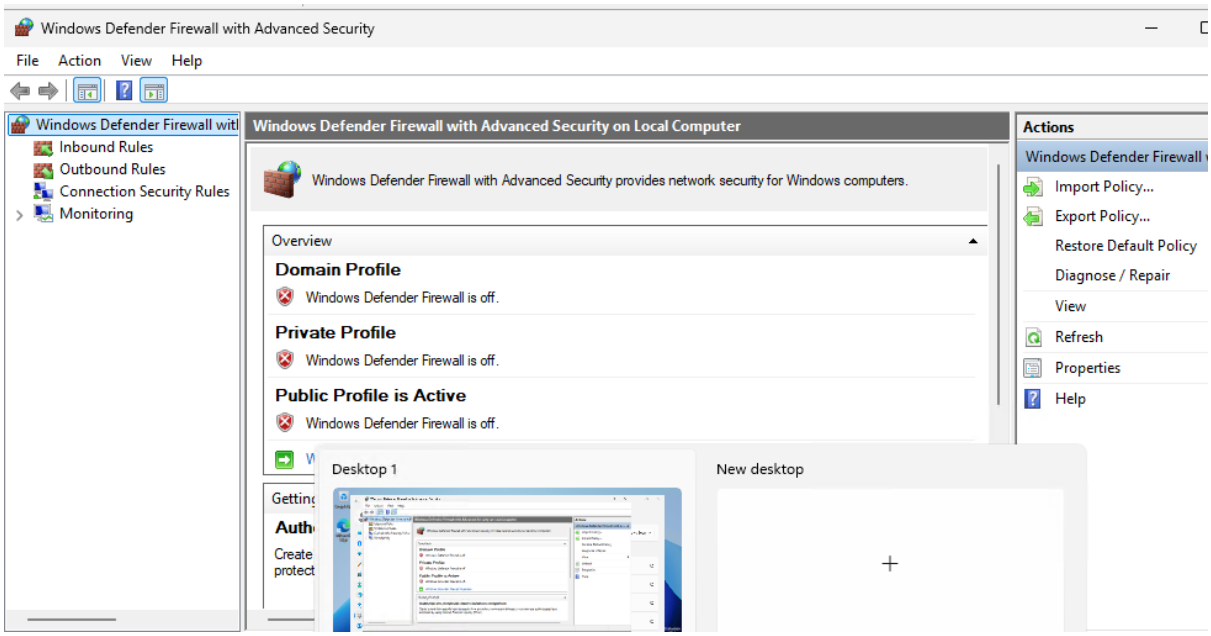
Firewall Considerations: Real World vs Lab Environment

In a real-world production environment, the firewall should always be enabled. In that situation, we need to carefully follow best practices regarding ports, meaning we must decide exactly which ports need to be opened (enabled), and which ports should remain closed (disabled), based on what services the server actually needs to run.

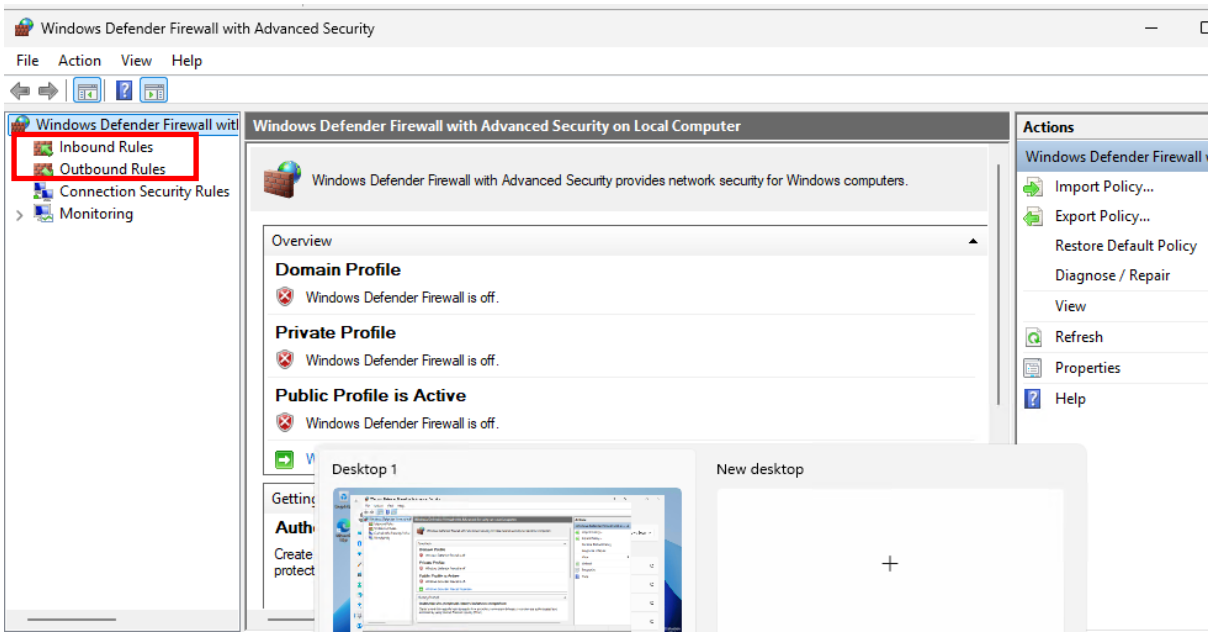
However, in this particular lab session, since we are only practicing and experimenting in a safe, isolated environment, we simply disable the firewall completely. This makes it easier for us to focus on learning Active Directory itself, without needing to also configure detailed firewall rules at the same time.

Note: Disabling the firewall is acceptable for lab and practice purposes, but this should never be done in a real, production server environment, as it would create serious security risks.





Those are the inbound rules and outbound rules



Adding a New Firewall Rule

If we want to add a new inbound rule (controlling traffic coming into the computer) or a new outbound rule (controlling traffic leaving the computer), we can simply click on the "New Rule" option, found on the right-hand side of the Windows Firewall management screen. This allows us to manually create a custom rule.

Understanding Windows Firewall Profiles

Windows Firewall uses three separate network profiles, called Domain, Private, and Public. These profiles allow the firewall to automatically adjust its security level, depending on how trustworthy the currently connected network is considered to be.

1. Domain Profile

This profile becomes active whenever a computer is connected to a network that is managed by a corporate network's domain controller, meaning it is part of an Active Directory environment.

- **Trust Level:** Highest trust.
- **Behavior:** Since the network is assumed to be secure and properly managed by IT administrators, the firewall allows corporate management traffic, background services, and file sharing to run smoothly, without unnecessary restrictions.

2. Private Profile

This profile is intended for home networks, or trusted small business networks, where the user themselves has full control over which devices are connected.

- **Trust Level:** Medium trust.
- **Behavior:** This profile enables Network Discovery, which allows our computer to see, and be seen by, other trusted devices on the same network, such as home printers or shared storage drives.

3. Public Profile

This profile is used by default whenever we connect to an untrusted public network, such as Wi-Fi hotspots found at airports, hotels, or coffee shops.

- **Trust Level:** Lowest trust (considered untrusted).
- **Behavior:** The firewall switches into maximum security mode. It completely disables Network Discovery, and it blocks other computers connected to the same public Wi-Fi network from being able to discover or access our device in any way.

Default Behavior vs Custom Rules

Whenever we install a new server role, such as Active Directory or DNS, onto a Windows Server, the operating system automatically creates and enables a set of basic, pre-defined firewall rules, which are specifically required for that particular service to function correctly.

However, even with these automatic rules already in place, system administrators still often need to manually create additional custom rules, for the following reasons:

1. Hardening Security

Custom rules can be used to restrict access even further, based on specific IP addresses or specific networks. For example, an administrator might configure a rule that only allows the IT Department's own subnet to access the server through Remote Desktop, blocking everyone else from doing so.

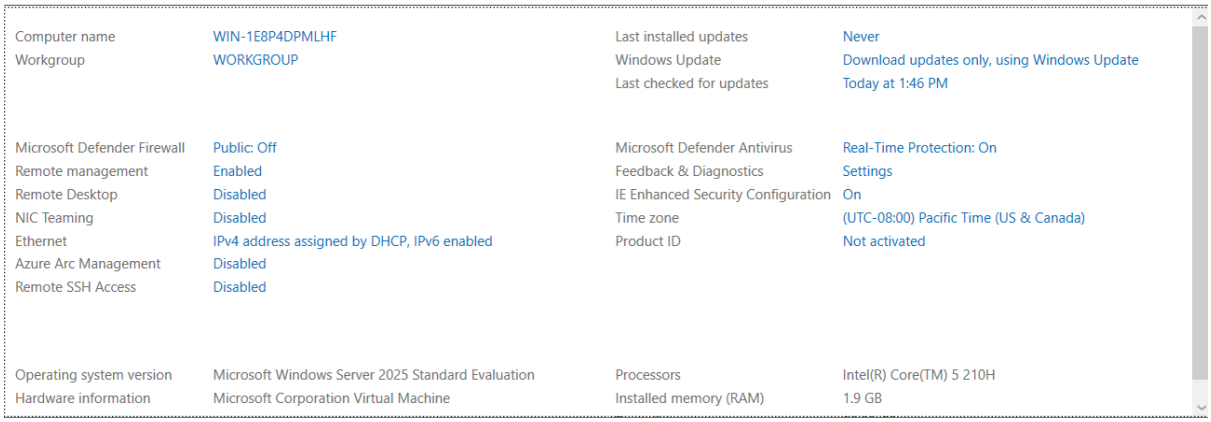
2. Supporting Third-Party Applications

Custom rules are also needed to allow network traffic for non-Microsoft software that the company might be using, such as accounting databases, HR systems, or backup tools. Since these are not built-in Windows services, they do not automatically receive their own pre-defined firewall rules, so an administrator must create one manually.

3. Network Troubleshooting

Sometimes, custom rules are needed simply for troubleshooting purposes. For example, a protocol like ICMP (commonly known as "Ping") is actually blocked by the firewall by default. An administrator may need to manually create a rule to allow ICMP traffic, so that they are able to properly test and confirm network connectivity between different clients and servers.

Server Basic Settings Overview



This screenshot shows the basic settings screen of a Windows Server, right after installation. It gives us a quick summary of important settings, such as the computer name, workgroup, firewall status, remote access settings, time zone, and hardware information, all in one place.

Remote Desktop (RDP) vs Remote Management

There are two common ways for an administrator to remotely access and control a server: Remote Desktop, and Remote Management. Even though both allow remote access, they work very differently, and are used for different purposes.

1. Remote Desktop (RDP)

Remote Desktop provides a full Graphical User Interface (GUI) of the remote server, meaning we see and interact with the server's actual desktop screen, exactly as if we were physically sitting in front of that machine ourselves.

- **Primary Use:** This is mainly used for manual configuration tasks, running graphical (GUI-based) applications, and visually troubleshooting problems on the server.
- **Default Port:** TCP port 3389.

2. Remote Management

Remote Management, on the other hand, allows administrators to manage and configure a server entirely in the background, without ever opening a full visual desktop session. Instead, it connects using command-line tools, such as PowerShell, or through remote management consoles, such as Server Manager or Windows Admin Center.

- **Primary Use:** This is mainly used for automation tasks, running scripts across many servers at the same time, and monitoring the overall health of a server, without using up heavy network bandwidth.
- **Default Ports:** TCP port 5985 (for HTTP) and TCP port 5986 (for HTTPS), both used through a system called WinRM (Windows Remote Management).

What is an NTP Server?

NTP stands for Network Time Protocol. An NTP Server is a specialized type of server, whose job is to synchronize the clocks of all computers, servers, and network devices within a network, making sure they all display the exact same, correct time.

Why is NTP so important for Active Directory?

Within a Windows Domain environment, having accurate, synchronized time across all devices is absolutely mandatory, for both security and authentication purposes.

- Windows uses a protocol called Kerberos for its authentication process.
- If there is a noticeable time difference, known as "Time Skew," between the Domain Controller (DC) and a client PC, of more than 5 minutes, then authentication will actually fail completely, and users will not be able to log into their computers at all.
- Because of this strict requirement, the Active Directory Domain Controller automatically takes on the role of the NTP Server for the entire local network, making sure that all client devices remain perfectly synchronized with each other at all times.\

How to connect AD via remote desktop (RDP)

To connect to our Active Directory server remotely, using Remote Desktop, we follow these steps:

1. First, we need to know the IP address of the Active Directory server.
2. Then, on the host computer (our laptop), we open the Remote Desktop Connection application, and type in that IP address.
3. We then enter the correct credentials, meaning the username and password of an account with access to that server.
4. We click "Yes" if a security certificate warning appears, and we are now successfully connected to the Active Directory server through RDP.

Important: Understanding APIPA Addresses

When we do not manually assign a static IP address to our server, Windows will automatically assign a temporary IP address instead, from a special range starting with 169.254.x.x. This type of automatically assigned address is called an APIPA address, which stands for Automatic Private IP Addressing.

Why does this happen with an Internal Switch?

When we use an Internal Switch in Hyper-V, there is no DHCP server available in that environment, to properly hand out real IP addresses. Because of this, Windows automatically creates its own temporary APIPA address instead, just so the network adapter has some kind of address to use.

Why is an APIPA address a problem for Active Directory?

An Active Directory server acts as the central "hub" of the entire network. However, an APIPA address is not stable, it can actually change unexpectedly, for example during a server restart, or due to other events. If this were to happen, client computers would suddenly be unable to find the server

anymore, and the entire network would effectively go down, since none of the clients would know the new address to connect to.

What happens if we use an External Switch instead?

If we use an External Switch instead of an Internal Switch, our virtual machine would be able to reach our actual physical router, and the router itself could hand out an IP address through DHCP. In that case, the APIPA issue would not occur, since a real IP address would already be available.

So why do we still need a Static IP address?

Even though using an External Switch with DHCP avoids the APIPA problem, the IP address received through DHCP from the router can still technically change at any time, since DHCP addresses are dynamic by nature.

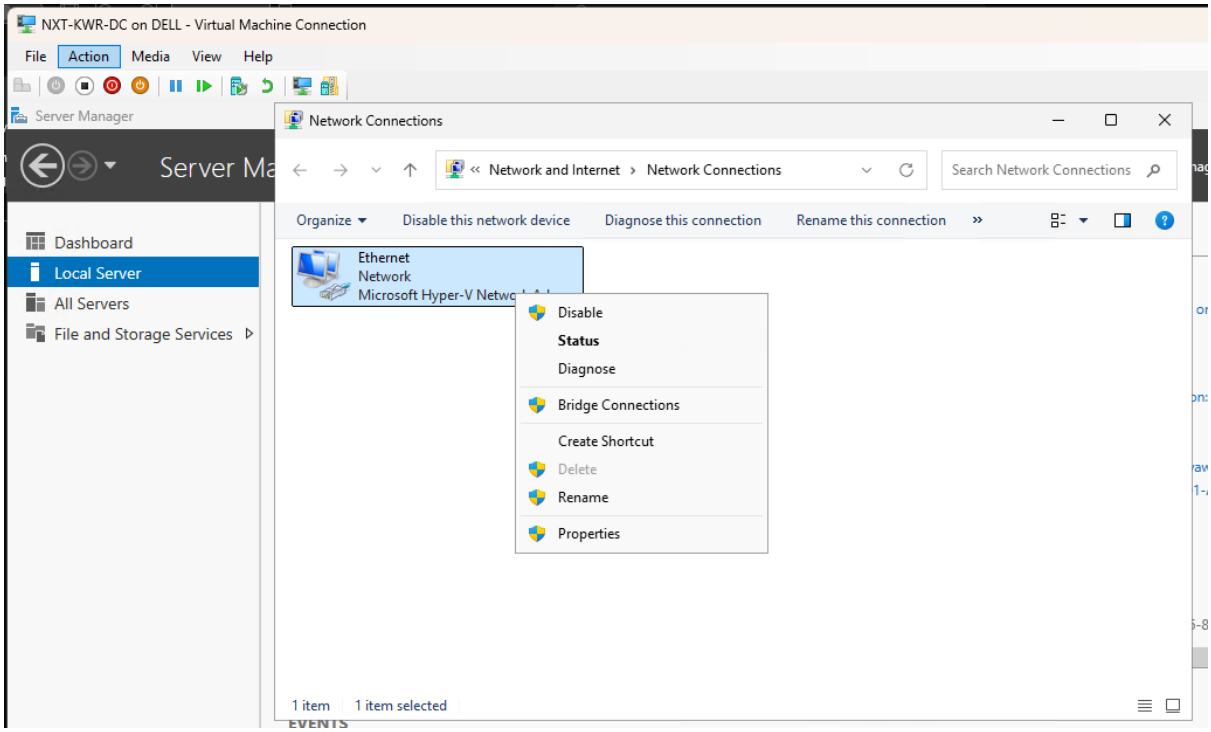
However, since an Active Directory server absolutely needs to keep the exact same address at all times, in order to remain reliable, we must manually assign it a Static IP address instead, one that will never automatically change, no matter what happens.

Assigning a Static IP Address to Our Server

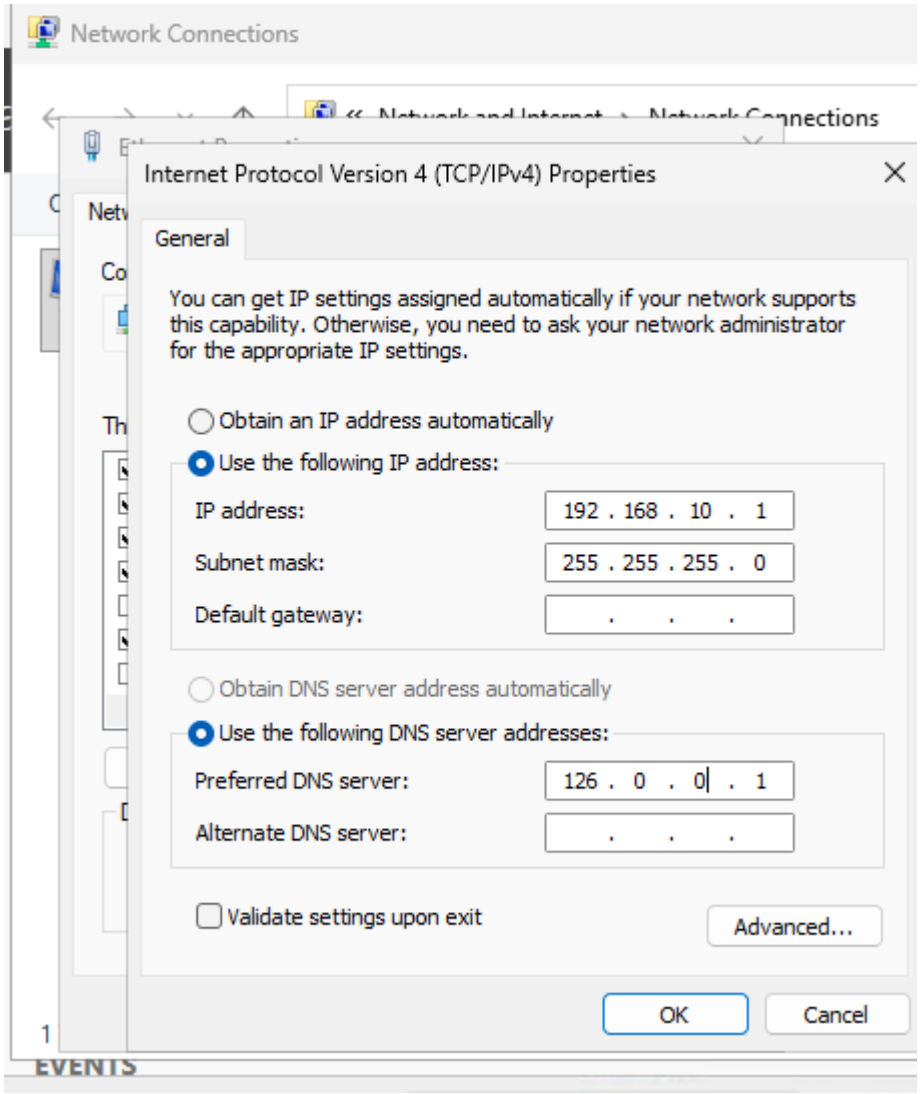
To assign a static IP address, we can follow this path:

Control Panel → Network and Internet → Network Connections

Alternatively, there is a faster shortcut available: press **Windows key + R**, then type **ncpa.cpl**, and press Enter. This will take us directly to the Network Connections screen, without needing to navigate through the Control Panel manually.



Select internet **Internet Protocol Version 4 (TCP/IPv4) and go properties**



Assigning a Static IP Address (Example Configuration)

In the screenshot above, we can see the Internet Protocol Version 4 (TCP/IPv4) Properties window, which is where we manually configure a static IP address on our server.

Here, the option "Use the following IP address" is selected, and the following details are entered:

- **IP address:** 192.168.10.1
- **Subnet mask:** 255.255.255.0
- **Preferred DNS server:** 127.0.0.1

Why We Use 127.0.0.1 for DNS Settings

The IP address 127.0.0.1 is known as the Loopback Address, or more commonly, "Localhost." This special address always refers back to the same machine itself, no matter which computer is using it.

The Role of DNS in Active Directory

Active Directory cannot function properly without DNS. This is because the DNS server is responsible for storing the exact locations of all domain services within the network. Because of this

close relationship, whenever we promote a server to become a Domain Controller, the DNS Server role is also automatically installed on that very same machine, alongside Active Directory.

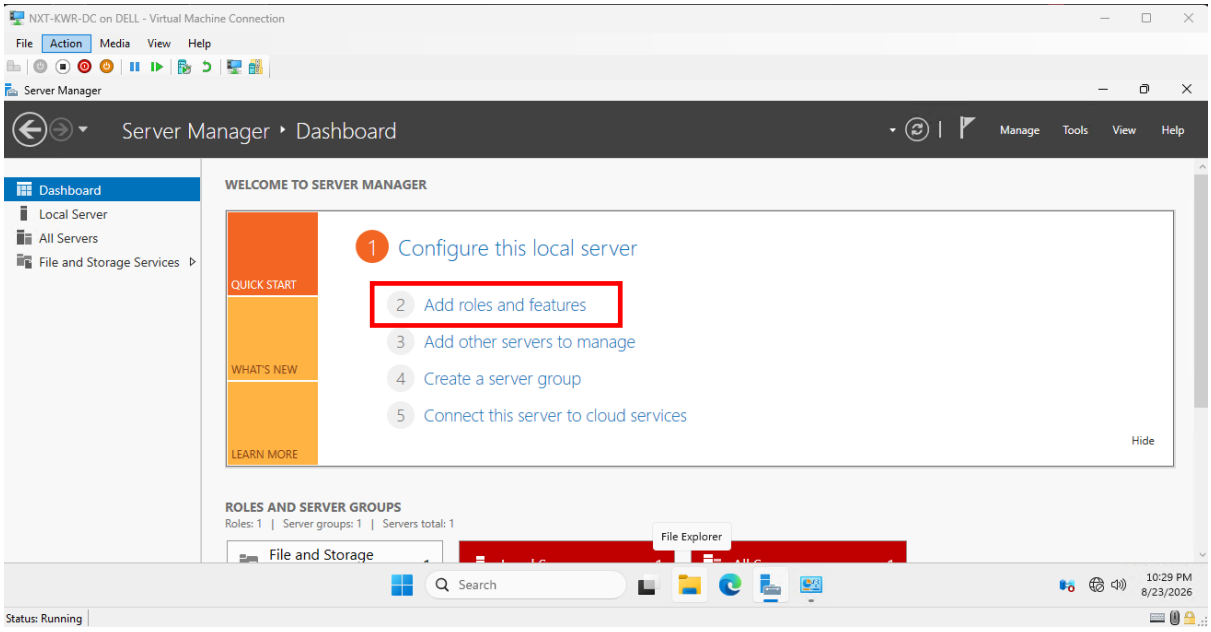
Why Do We Use the Loopback Address (127.0.0.1) for DNS?

1. Self-Resolution

By setting our server's own DNS server address to 127.0.0.1, we are essentially telling Windows Server: "Use your own local DNS database, to resolve any DNS queries you receive." Since the DNS Server role is already installed directly on this same machine, the server does not need to look anywhere else for DNS information, it can simply refer to itself, using the loopback address, since it already contains its own complete DNS records.

This setup makes the server more reliable and self-sufficient, since it does not depend on any other external DNS server, in order to resolve its own domain-related queries.

How to add Active directory in server manager



When you click the Add roles and Features section now you can see below options

Management, and Computer Management. Having these shortcuts readily available on the desktop will save us a lot of time, as we continue working with Active Directory.

Next Step: Setting Up DNS

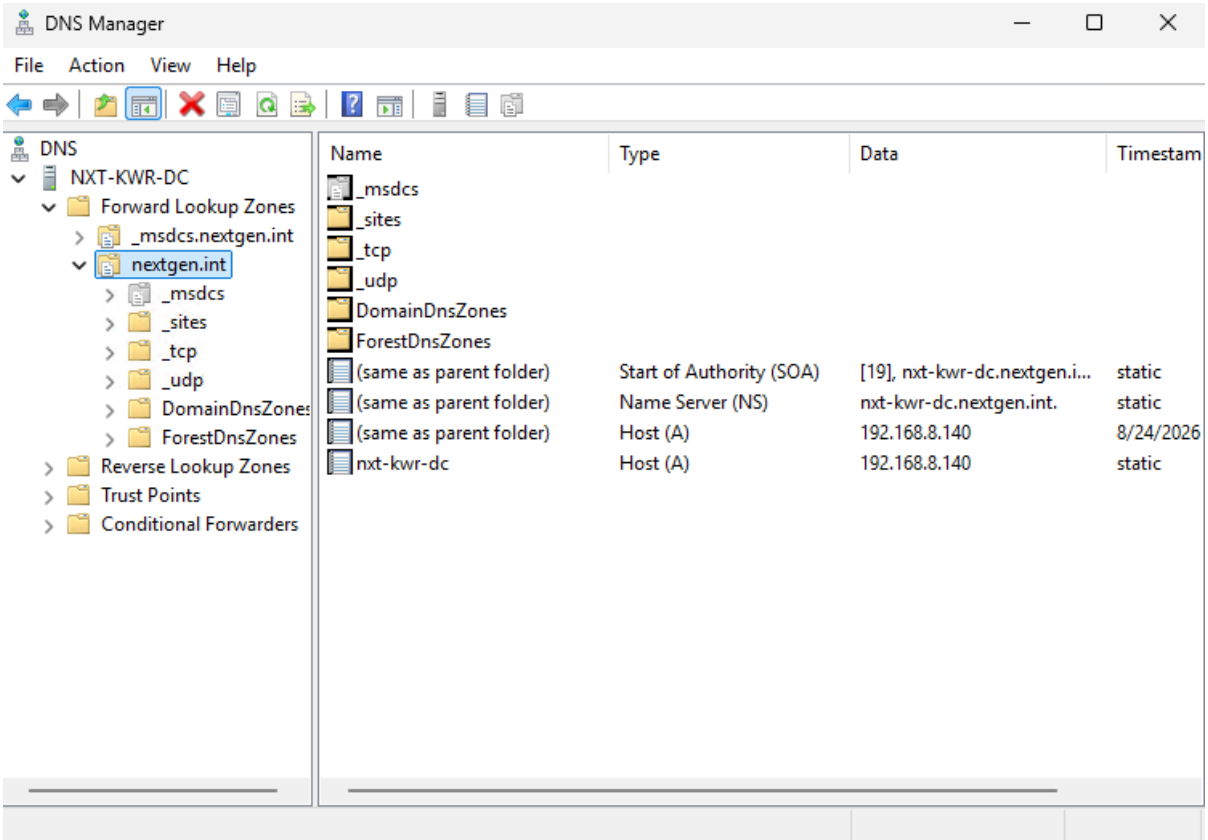
Now that Active Directory has been successfully installed and configured, the very next important step is to properly set up and verify the DNS server, since Active Directory depends so heavily on DNS in order to function correctly.

DNS Zones, Organizational Units, Users, Groups, and Client Domain Join

Forward Lookup Zone vs Reverse Lookup Zone

Inside DNS Manager, there are two different types of zones that work in opposite directions to each other:

- Forward Lookup Zone: Translates a name into an IP address (Name-to-IP).
- Reverse Lookup Zone: Translates an IP address back into a name (IP-to-Name).



Inside the Forward Lookup Zone

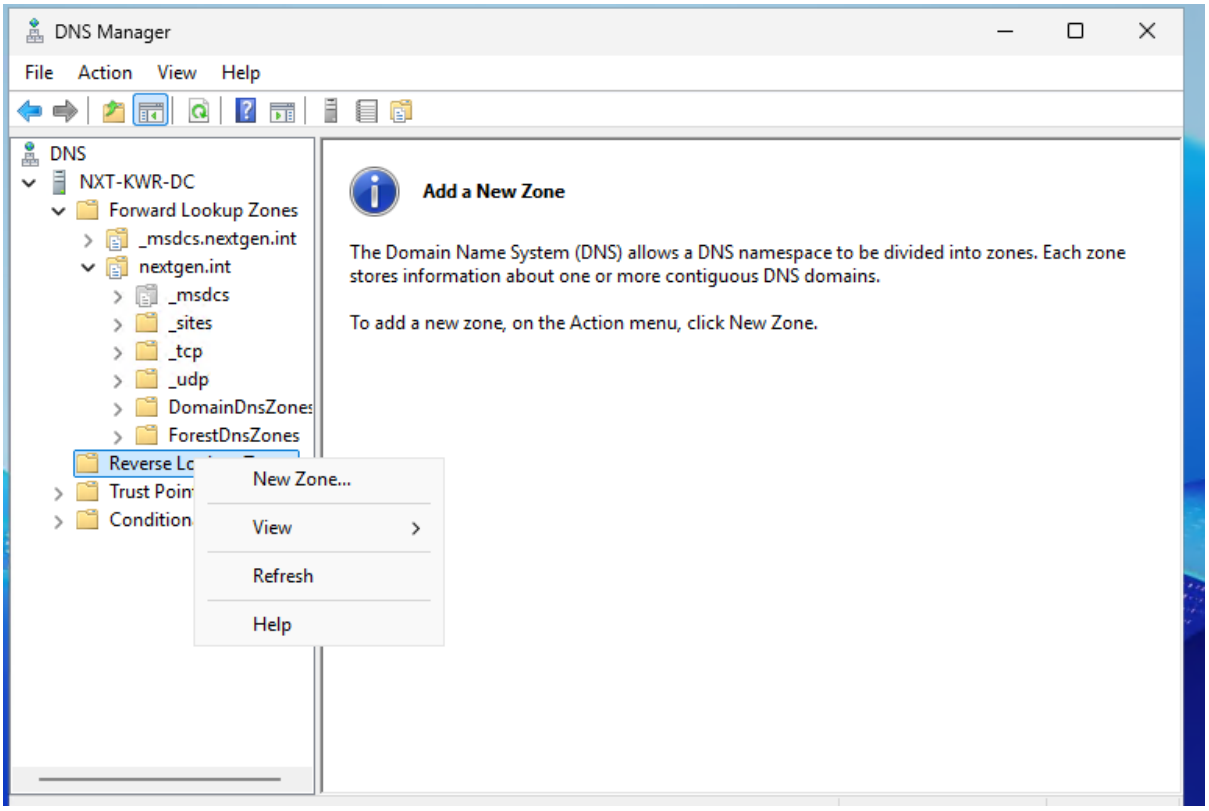
Looking at the records shown in the Forward Lookup Zone above, we can identify a few important record types:

- **Start of Authority (SOA):** This record identifies the primary, authoritative DNS server (in this case, next_gen_ad.nextgen.int), which is responsible for maintaining the health and synchronization settings of this specific zone.
- **Name Server (NS):** This record specifies exactly which authorized domain controllers are hosting the DNS service, for the nextgen.int namespace.

- **Host (A) Records:** These records map human-readable hostnames directly to a static IPv4 address (in this case, 192.168.10.1). The parent record allows communication using just the domain name itself (nextgen.int), while the specific (next_gen_ad) record maps the individual server's hostname.

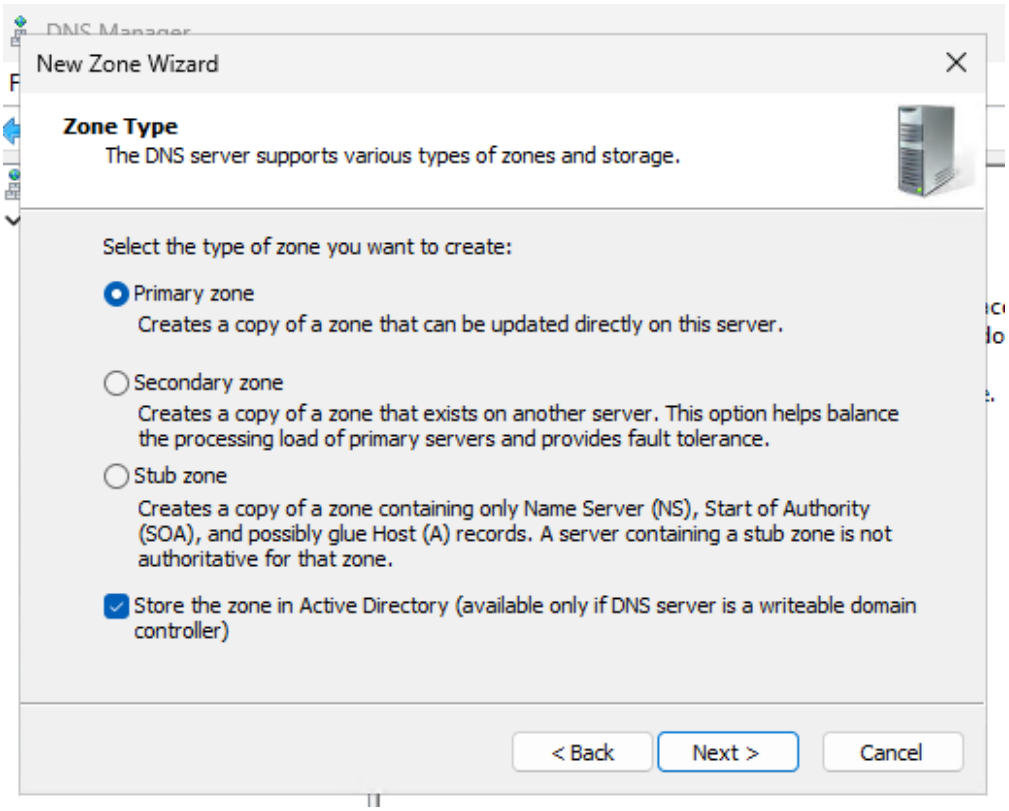
Inside the Reverse Lookup Zone

Unlike the Forward Lookup Zone, a Reverse Lookup Zone is not created automatically by default when we set up Active Directory. Because of this, we need to manually create a new Reverse Lookup Zone ourselves, if we want this functionality available on our network.



To begin creating one, we right-click on the "Reverse Lookup Zones" folder inside DNS Manager, and select "New Zone," which launches the New Zone Wizard.

Choosing the Zone Type

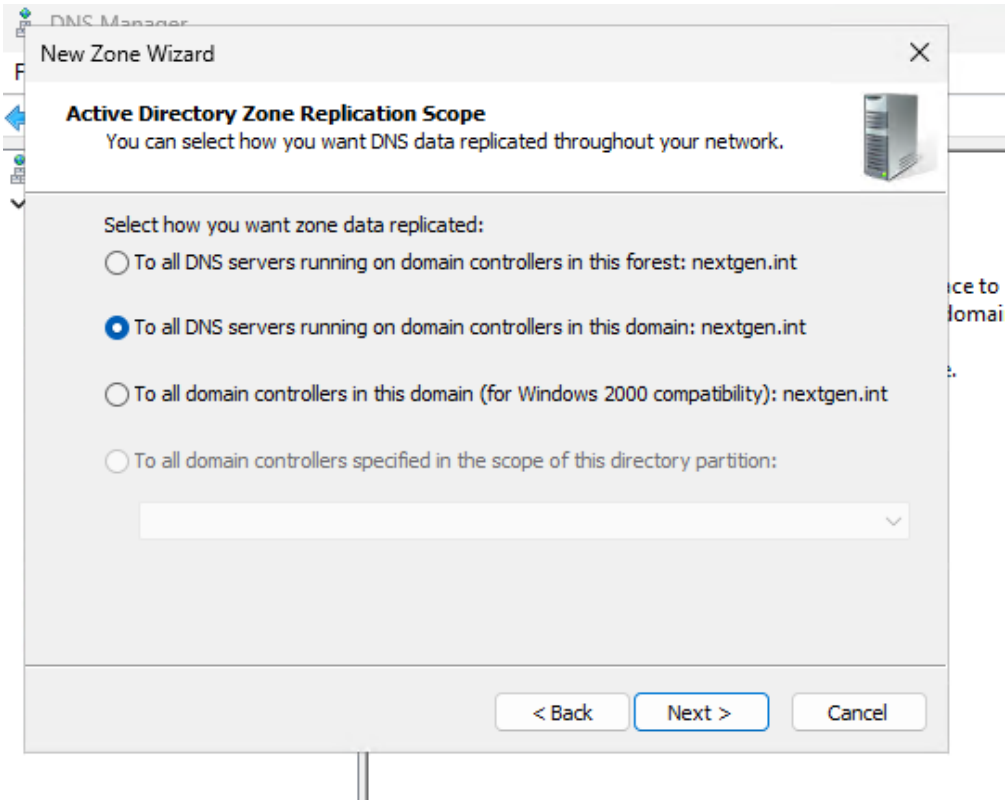


During this wizard, we are asked to choose a zone type. Here is what each option means:

- **Primary Zone:** This is the main, original copy of the zone, also known as the Master Copy. It has full authority to change the data within this zone, and to add new IP information. Since there are currently no other DNS servers in our lab environment, we must definitely select "Primary zone" here.
- **Secondary Zone:** This acts as a "backup server," which simply keeps a copy of the data from the main server. It cannot enter any new data itself; it can only read the existing data (Read-Only).
- **Stub Zone:** This is a very small record, used only to find out who the other main servers are on the network (containing only SOA and NS records), without actually storing the entire zone's data.

To understand a Stub Zone better, imagine that our company, nextgen.int, buys another company called abc.int. Now, the networks of these two companies need to be able to recognize each other. If we create a stub zone for the company "abc" inside our nextgen server, then whenever a user on the nextgen network tries to visit a website or server belonging to abc, our server can directly find the location (path) of abc's main server, and correctly send the request there.

Replication Scope: How Far Should the Data Spread?



Since we are creating this zone as Active Directory-Integrated, we also need to decide the Replication Scope, meaning how widely this DNS data should be copied and shared across our network. Here is what each option means:

- **Forest-Wide Replication:** This replicates the DNS data to every single domain controller running DNS, across the entire Active Directory Forest. This is best suited for global enterprises that have multiple sub-domains.
- **Domain-Wide Replication (the option we selected):** This limits replication exclusively to domain controllers that are hosting the DNS role, within our own specific nextgen.int domain boundary. This helps optimize network traffic, and keeps data properly isolated.
- **Legacy Compatibility:** This option is provided purely for backwards compatibility with very old operating systems (such as Windows 2000), and it is generally considered deprecated in modern network designs.

Understanding Forest-Wide Replication with an Example

Let's imagine that our company is located in Sri Lanka, as the head office, and it has two additional branches, one in India and one in Australia, each with their own separate IP address and network rules.

- The main domain in Sri Lanka: nextgen.int (this acts as the parent domain).
- The branch in India (a Child Domain): in.nextgen.int
- The branch in Australia (a Child Domain): au.nextgen.int

The entire system, formed when all three of these domains are combined together, is what we call a Forest.

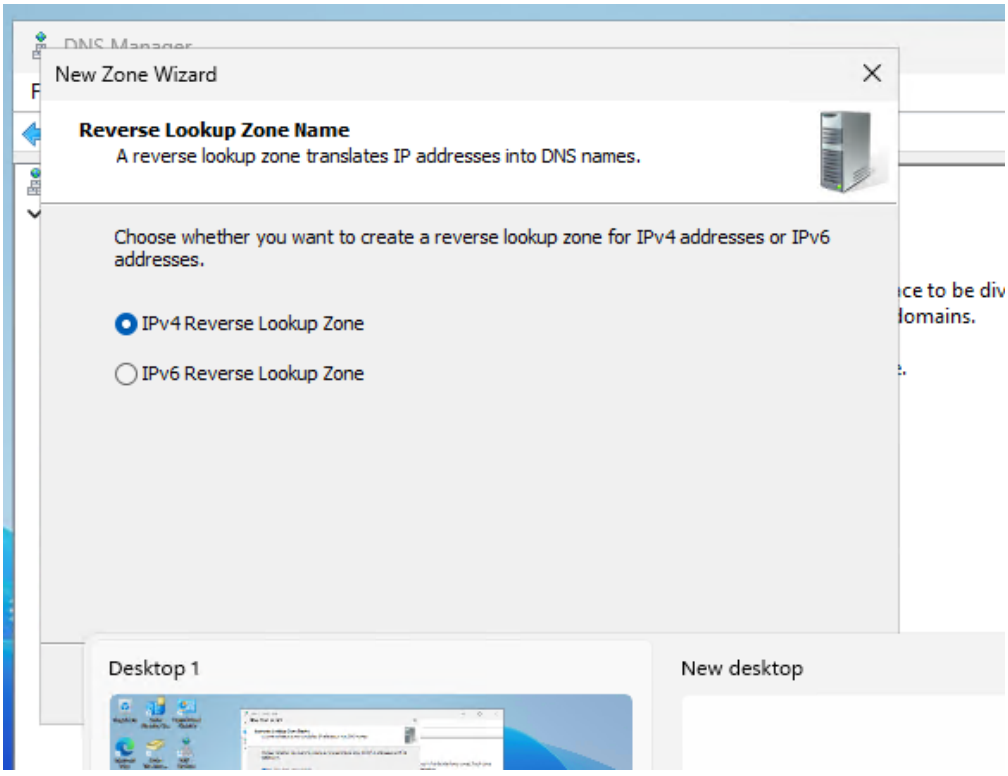
If we were to enable Forest-Wide Replication here, then any Reverse Lookup settings and IP details that we create on the Sri Lankan server would automatically be replicated across to the servers located in India and Australia as well.

Understanding Domain-Wide Replication with an Example

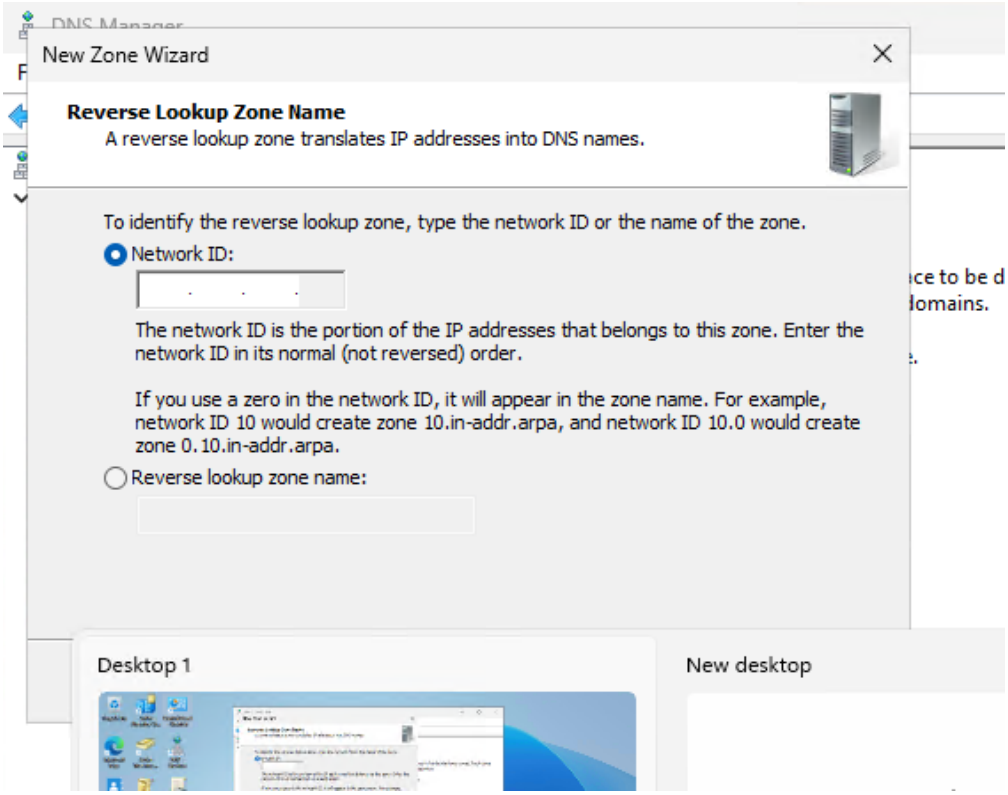
Now, let's imagine instead that we want to keep our data in Sri Lanka strictly within Sri Lanka, without sending it to the servers in India or Australia at all.

If we set the replication scope to Domain-Wide instead, then this Reverse Lookup data would only be copied within the nextgen.int domain boundary. If we were to create additional backup Domain Controllers within Sri Lanka itself, the data would only replicate to those particular servers, and it would never be sent onward to the in.nextgen.int or au.nextgen.int servers located in India or Australia.

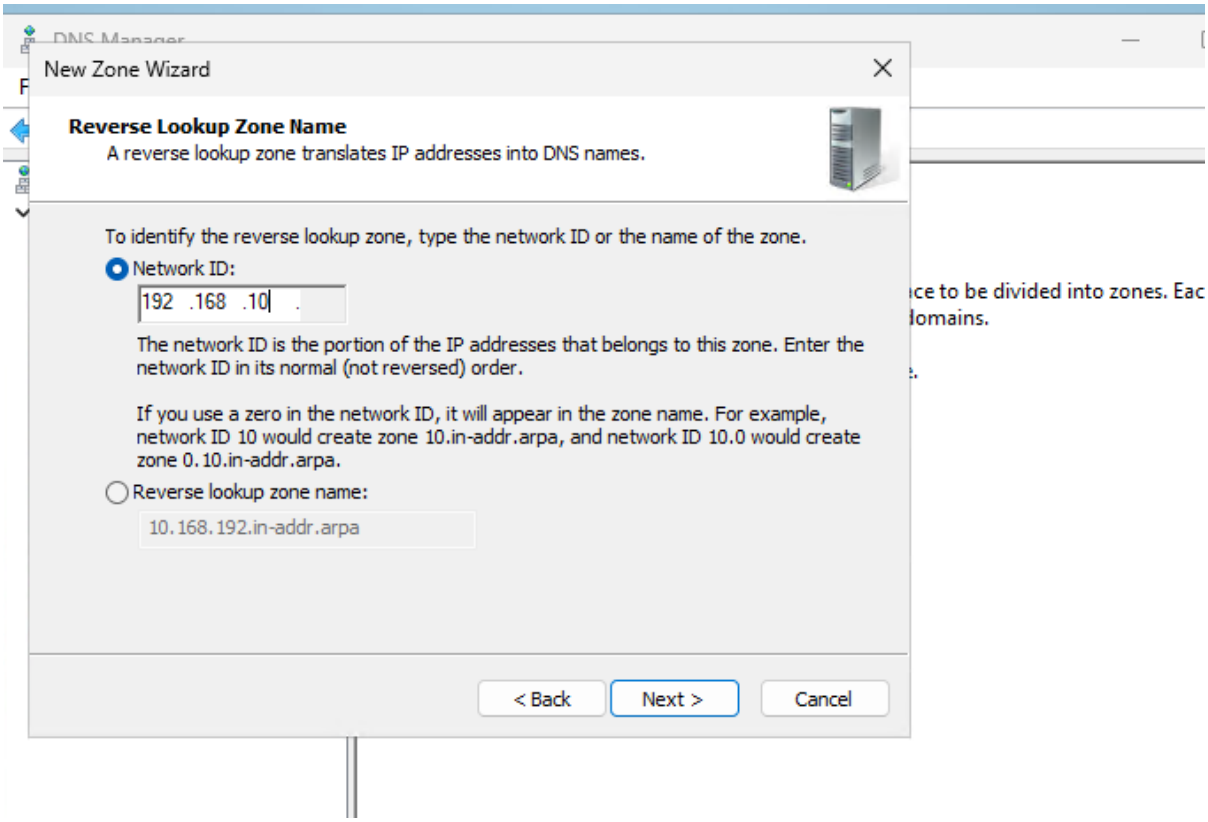
Setting the IP Version and Network ID



Next, the wizard asks us to choose between creating an IPv4 or an IPv6 Reverse Lookup Zone. In our lab, we use IPv4.



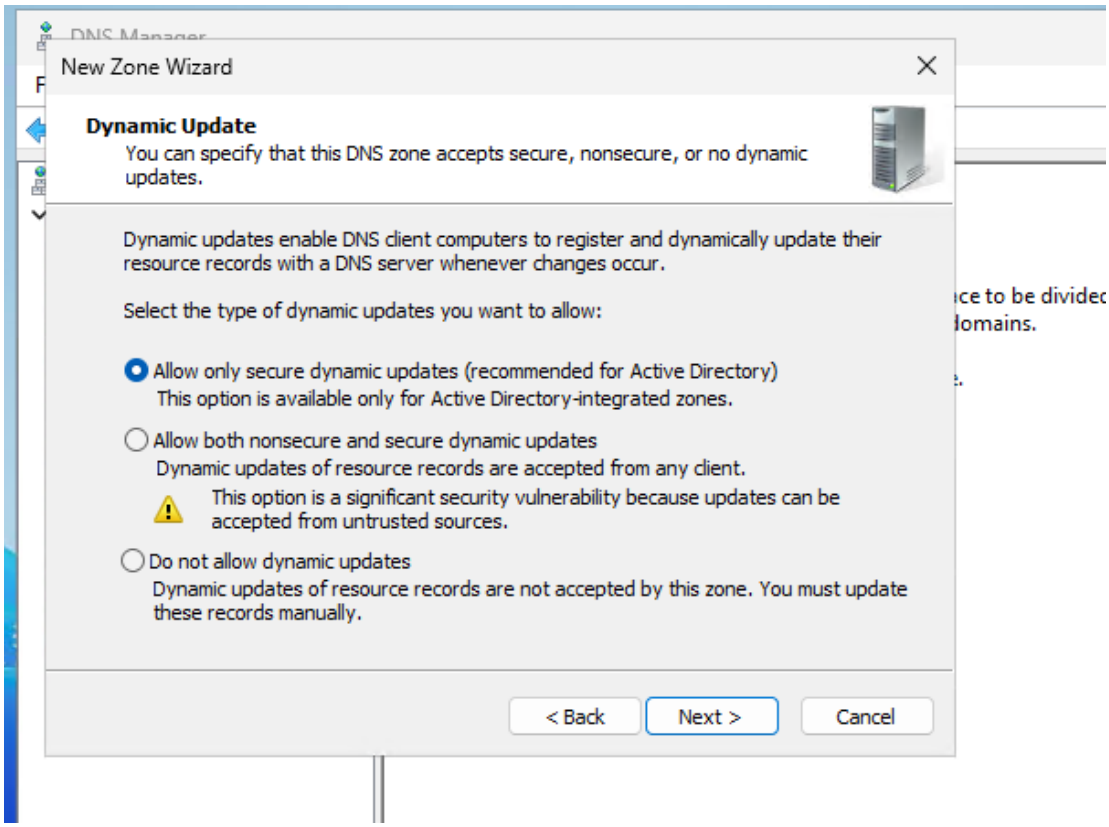
On this screen, we need to type in the Network ID, which tells the DNS server exactly which IP range this particular reverse lookup directory should apply to.



In our example, we entered 192.168.10 into the Network ID field. Based on this input, the system automatically generated the zone name as 10.168.192.in-addr.arpa. This particular format, where the

numbers of the IP address are reversed, is the standard DNS format used for mapping IPv4 addresses back to hostnames.

Choosing the Dynamic Update Method



Next, we are asked to choose how this DNS zone will accept dynamic updates from client computers. Here is what each option means:

- **Allow only secure dynamic updates (the option we selected):** This is considered the gold standard for Active Directory. It makes sure that only computers which are already authenticated within the domain are able to register or update their own DNS records. This helps prevent something called "DNS spoofing," where an unauthorized device might try to overwrite a server's record.
- **Allow both nonsecure and secure updates:** This option opens up the server to significant security vulnerabilities, since any device on the network, even unmanaged or potentially malicious ones, would be able to modify DNS entries freely.
- **Do not allow dynamic updates:** This disables automation entirely, meaning the administrator would need to manually enter every single Pointer (PTR) record themselves. This becomes very inefficient as a network grows larger.

By selecting Secure Updates, our Reverse Lookup Zone becomes Active Directory-integrated, which provides seamless and secure IP-to-Hostname mapping, for every device that is a member of our domain.

Why Do We Configure a Reverse Lookup Zone?

- **Security & Mutual Authentication:** Active Directory uses reverse lookup to help validate the identity of connecting devices. When a client requests access, the server can verify that

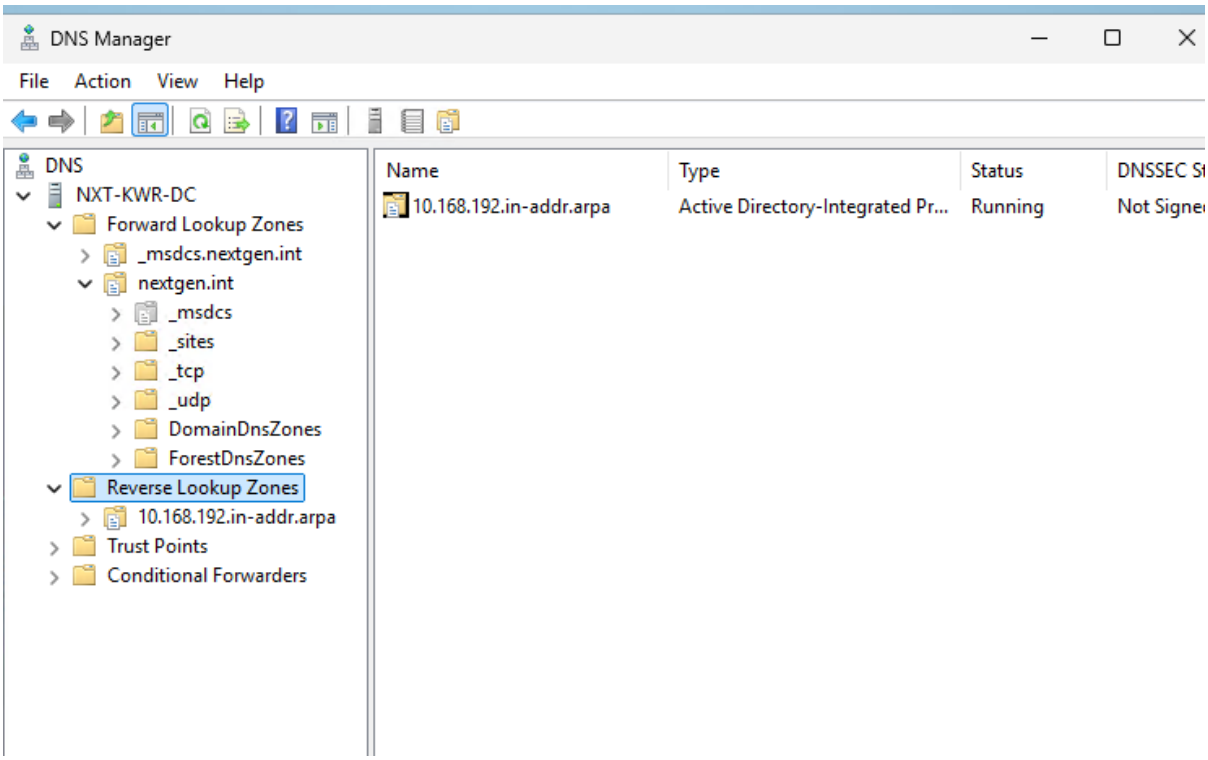
the client's IP address genuinely matches its registered hostname, using a Pointer (PTR) record. This helps prevent IP spoofing attacks.

- **Network Diagnostics & Troubleshooting:** System administrators rely heavily on tools such as nslookup [IP] or ping -a [IP], to diagnose connectivity issues. Without a reverse lookup zone in place, these tools would fail to resolve and display the human-readable computer names, from log files and network traffic.

How This Works in Our Infrastructure

Step A: Establishing the Authority. By creating a Primary, Active Directory-Integrated Zone, and specifying the Network ID 192.168.10, we created an authoritative container named 10.168.192.in-addr.arpa. This tells our Domain Controller (next_gen_ad) that it is responsible for managing all IP-to-name translations, within that specific network range.

Step B: The "Secure Dynamic Update" Gatekeeper. During setup, we enforced "Allow only secure dynamic updates." This acts as a security filter, meaning the DNS server will explicitly reject any machine trying to register an IP address, unless that machine is already an authenticated, official member of the Active Directory domain.



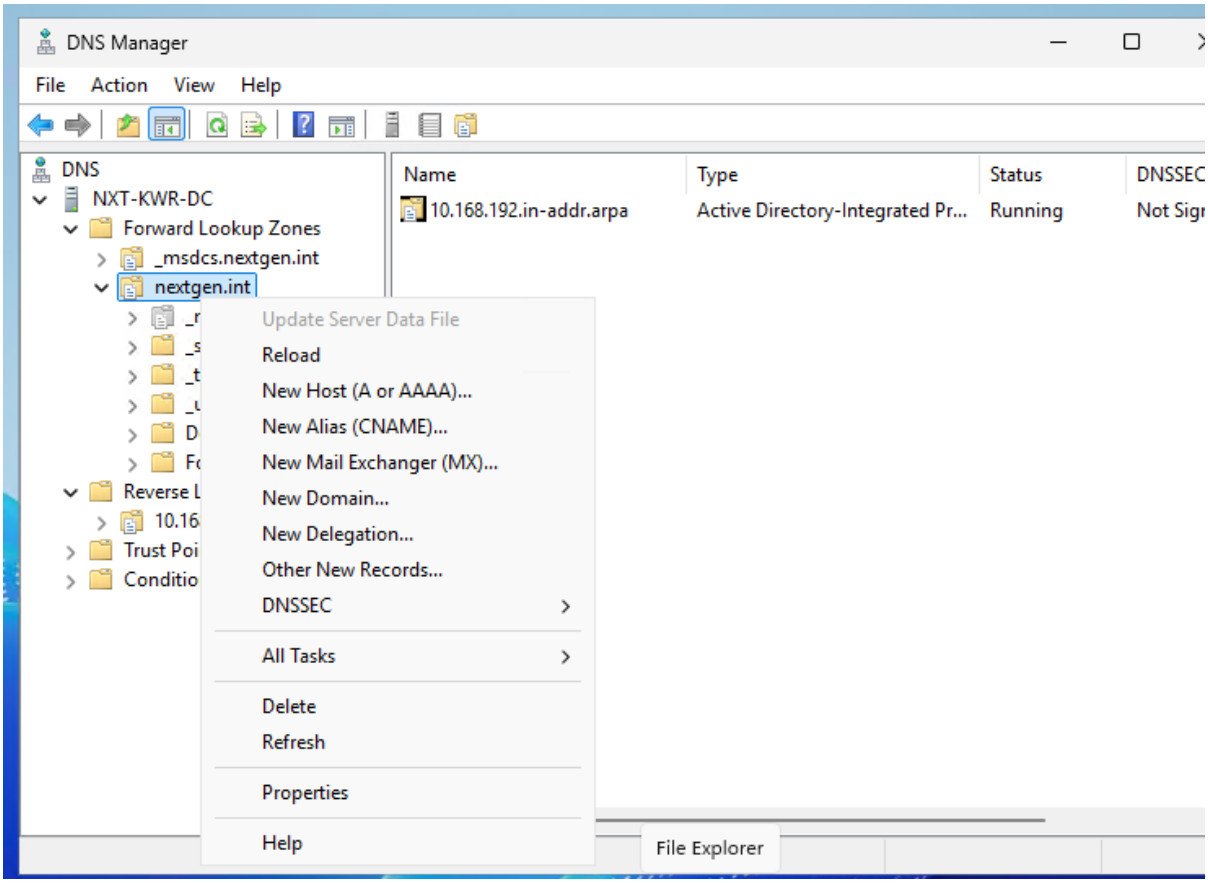
The screenshot above confirms our newly created Reverse Lookup Zone, 10.168.192.in-addr.arpa, now showing as an Active Directory-Integrated, Running zone.

The Automated Registration Flow (How It All Comes Together)

- Client Setup: The Windows client machine is configured with a static IP within the subnet, and its Preferred DNS setting points to the server's IP address (192.168.10.1).
- The Domain Join (Trigger): The client is formally joined to the nextgen.int domain, using domain administrator credentials.

- Trust & Registry: Upon restarting, the domain-joined client authenticates securely with the Domain Controller. Since it is now a trusted member of the domain, the client sends a registration message, essentially saying: "My hostname is Client-01, and my IP address is 192.168.10.X."
- PTR Record Creation: The DNS server accepts this trusted request, maps it into the reverse zone, and automatically writes a Pointer (PTR) Record, linking that specific IP address back to the computer's name. No manual IP entries are required from the administrator at all.

Adding New Records and Domains in DNS Manager



By right-clicking on our domain (nextgen.int) inside DNS Manager, we get access to several important options for adding new records and namespaces.

Operational Logic for Adding a New Domain Workspace

Adding a new domain namespace depends strictly on the intended logical directory architecture we want to build:

Scenario A: Creating a Sub-Domain (Child Domain)

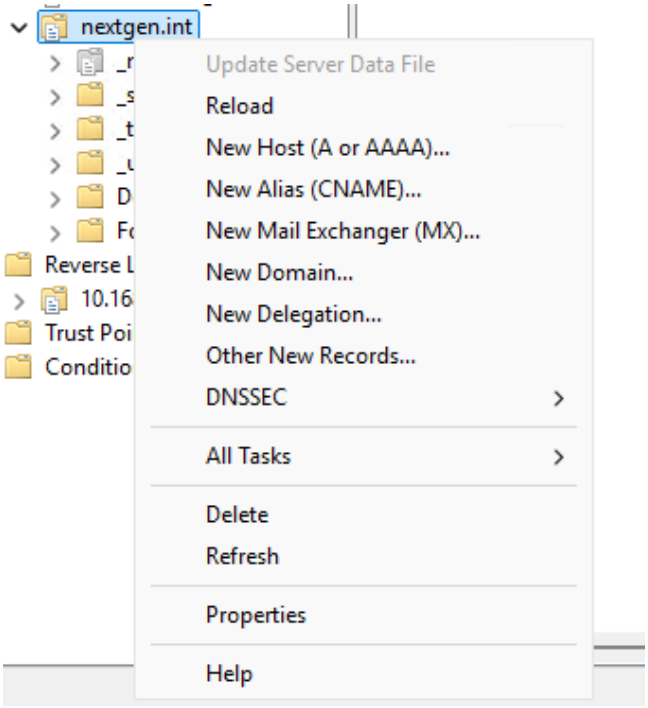
- Action: Right-click the existing root domain (nextgen.int), and select "New Domain."
- Result: This creates a contiguous namespace (for example, sales.nextgen.int), which inherits settings from, and exists inside, the logical tree boundary of the parent domain.

Scenario B: Creating an Independent Domain (New Root Tree)

- Action: Right-click the Forward Lookup Zones container directory, and choose "New Zone."
- Result: This initializes a completely separate Active Directory-Integrated Primary Zone database (for example, mycompany.local), establishing a distinct, non-contiguous namespace, split apart from the nextgen.int topology.

Key Menu Options Explained

- **New Host (A or AAAA):** Used to manually map a device's hostname to its static IPv4 (A) or IPv6 (AAAA) address. This is critical when adding infrastructure items like network printers, switches, or non-Windows servers, which cannot register themselves automatically.
- **New Alias (CNAME):** Creates a canonical name, or alias (essentially a nickname), which points back to an existing Host (A) record. This is useful for hosting multiple services on a single server, under different names (for example, pointing ftp.nextgen.int toward the main server).
- **New Mail Exchanger (MX):** Specifies the mail server responsible for accepting email messages, on behalf of the nextgen.int domain namespace.
- **Properties:** Opens the zone's configuration properties, allowing us to modify core parameters, such as replication scopes, zone type shifts, and dynamic update security settings.



Automated Client Dynamic Registration into a Sub-Domain

Suppose we want to create a child domain called "sales.nextgen.int" inside nextgen.int, by right-clicking nextgen.int and selecting New Domain. Here is how client machines actually get registered into this new sub-domain, in a real production environment:

- DNS Configuration: The target client machine's TCP/IP properties are adjusted, to point to the master domain controller (192.168.10.1), as its Preferred DNS server.

- Domain Binding Change: In the Advanced System Settings of the client, the computer's domain target is changed from a workgroup, directly to the full child domain string: sales.nextgen.int.
- Result: After entering the correct directory administrative credentials, and restarting the machine, the client authenticates against the directory structure. The DNS server interprets its partition path, and automatically creates a dynamic Host (A) record named PC01, directly inside the sales container subtree.

Do We Need a New Reverse Lookup Zone for a New Sub-Domain?

This actually depends on whether the new sub-domain shares the same IP subnet as the parent domain, or uses a completely separate one.

- **Case A - Same/Shared IP Subnet (192.168.10.0/24):** No new Reverse Lookup Zone is required. The already-existing 10.168.192.in-addr.arpa zone handles all reverse pointer resolutions for this range. Even if a host belongs to the sales sub-domain, its dynamic PTR record still automatically maps into the existing zone.
- **Case B - A Different, Separate IP Subnet (192.168.20.0/24):** Yes, a second Reverse Lookup Zone must be manually created. Since the original zone is structurally limited to only handling the 192.168.10.x address range, it simply cannot translate information from a completely different subnet. The administrator must create a new Active Directory-Integrated zone, specifically targeting Network ID 192.168.20.

Understanding DNS Delegation

Imagine our main company is located in Colombo (nextgen.int), where we host our primary DNS server. Now, suppose we are opening a new company branch in Kandy, and assigning it a Child Domain named kandy.nextgen.int. To manage operations and maintain computers at the Kandy branch, a separate IT team, along with a dedicated DNS server, has been set up over there. In a scenario like this, we can use the New Delegation option:

- Action: From Colombo, we create a New Delegation named "kandy," inside nextgen.int. While configuring it, we tell our server: "The IP address of the Kandy server is 192.168.50.1. From now on, every computer record belonging to kandy.nextgen.int will be managed by the server located there."
- Outcome: The Kandy IT team now gets full autonomy to create or delete any record, such as PC01.kandy.nextgen.int, directly within their own server, without ever needing to request permission from the head office.
- Query Resolution: If someone queries the primary server in Colombo, asking for PC01.kandy.nextgen.int, the primary server will directly refer, or "delegate," that request onward to the Kandy DNS server.

DNS Security Extensions (DNSSEC)

DNSSEC is a suite of extension specifications, designed to secure the Domain Name System, by providing cryptographic authentication to DNS responses. It protects networks against cyber-attacks such as DNS Cache Poisoning, DNS Spoofing, and Man-in-the-Middle (MITM) redirection.